

POLITECNICO DI MILANO

Electronic Technologies and Biosensors Laboratory

Breathing Simulator: Final Project Report

Violet Group

Professor:

Prof. Chiara Veneroni

Project Supervisor:

Matteo Mentasti

Group Members

Francesco Amaglio

Martina Artuffo

Alessandro Carbone

Simone Ghini

Andrea Penserini

Cristian Petramale

Academic Year 2025/2026

August 25, 2026

Contents

1	Introduction	3
1.1	Background	3
1.2	Objectives	4
2	PCB Design	5
2.1	System Architecture	5
2.1.1	Logic and Data Acquisition	5
2.1.2	Motor Actuation	6
2.1.3	Power Management	6
2.2	Schematic Design	7
2.2.1	Power Section	7
2.2.2	ESCON 50/5 and Protection	8
2.2.3	MCU and EEPROM	9
2.2.4	Connectors	10
2.3	PCB Layout	10
2.3.1	Stackup and Manufacturing Choices	10
2.3.2	Connector Placement	11
3	3D Modeling and Assembly	12
3.1	CAD Models	12
3.1.1	Switch Supports	12
3.1.2	Syringe Supports	13
3.1.3	Motor Support	13
3.1.4	Encoder Support	14
3.1.5	PCB Support	14
3.1.6	SFM3304 Adapter	14
3.1.7	Pneumotachograph	15
4	Firmware	16
4.1	Architecture overview	16
4.2	Memory Organization and Data Structures	16
4.3	Interrupt Service Routines	19
4.3.1	Synchronous events	19
4.3.2	Asynchronous events	21
4.4	Bluetooth communication	22
4.4.1	Data Reception	22
4.4.2	Command Parser	23
4.4.3	Data Transmission	23
4.4.4	GUI synchronization	24
4.5	PID Architecture	24

5	GUI	26
5.1	Main goal	26
5.2	Communication management	26
5.3	State synchronization and input validation	27
5.4	Plot Rendering	28
5.5	Data logging abstraction	29
5.5.1	File I/O Abstraction	29
5.5.2	Event-Driven Synchronization	30
5.5.3	Data Integrity	31
6	Main Challenges and Solutions	32
6.1	Software Issues	32
6.1.1	Outdated Bluetooth Module	32
6.1.2	UART Communication Complications	32
6.2	Wrong Measurements for CAD Models	32
6.3	PCB Issues and Fixes	33
6.3.1	Main PCB Issues	33
6.3.2	Second Review and Design Improvements	33
7	Testing Phase	35
7.1	Calibrations	35
7.2	System Validation	35
8	Conclusions & Future Work	36
8.1	Conclusions	36
8.2	Future Work	37

1 Introduction

1.1 Background

The controlled reproduction of respiratory airflow patterns plays an important role in the development and bench testing of sensors, monitoring systems, and other biomedical devices designed to measure or interact with respiration. A breathing simulator provides a repeatable and controllable experimental platform through which different respiratory conditions can be reproduced without requiring the direct involvement of a human subject.

During spontaneous breathing, airflow alternates between inspiration and expiration as a consequence of cyclic changes in thoracic volume and pressure. Although quiet breathing can be approximated by a smooth periodic waveform, physiological respiratory signals are not perfectly sinusoidal and may present variations in amplitude, duration, the relative duration and timing of inspiration and expiration, and cycle-to-cycle regularity. Therefore, a breathing simulator intended for biomedical testing should be able to generate both simple and repeatable reference profiles, such as steady and sinusoidal flows, and more realistic respiratory patterns obtained from in vivo recordings.

The system developed within Project Violet is an adult breathing simulator based on the controlled displacement of a syringe plunger. The syringe acts as a variable-volume chamber, while its piston is driven by a brushless motor through a mechanical transmission system. The resulting piston displacement generates airflow at the syringe outlet, which can be controlled to reproduce different respiratory profiles. An incremental optical encoder monitors the actuator motion, while flow and differential-pressure sensors measure the pneumatic output of the system. A microcontroller coordinates actuator control, sensor acquisition, and the exchange of commands and measurement data with external software.

The project started from a pre-existing mechanical structure that had already been partially assembled. However, the initial prototype was not yet suitable for operation as a complete, reliable, and integrated breathing simulator. Several 3D-printed components presented structural, dimensional, or compatibility limitations and therefore required redesign and manufacturing. Moreover, the integration of the selected brushless motor and optical encoder required the development of dedicated mechanical adapters and supports. New components were consequently designed for the motor, encoder, limit switches, syringe, and electronic board, with the aim of improving mechanical stability, component alignment, and overall system integration.

Beyond the mechanical redesign, the project required the development of a custom printed circuit board, embedded firmware, and computer software. The firmware was developed to manage the actuator interfaces, monitor the mechanical motion through the encoder, perform timed sensor acquisition, and communicate with the external control application. The computer software provided the functions required to configure the system, select its operating modes, monitor the acquired signals, and support testing and debugging. These elements were combined into a single functional mechatronic platform integrating mechanical actuation, electronic control,

sensing, and data communication.

1.2 Objectives

The main objective of the project was to design and develop a functional adult breathing simulator capable of generating steady and sinusoidal airflow profiles, as well as reproducing respiratory patterns previously recorded in vivo. The simulator was intended to provide controlled and repeatable pneumatic outputs that could be used for the development and evaluation of respiratory sensors and monitoring systems.

From a mechanical perspective, the project aimed to adapt and improve the available prototype by redesigning and manufacturing the required 3D-printed supports and adapters. Particular attention was given to the correct positioning and alignment of the syringe, transmission components, brushless motor, incremental encoder, limit switches, and electronic hardware.

From an electronic perspective, the objective was to design and fabricate a custom printed circuit board capable of integrating the microcontroller, motor driver, encoder, flow and pressure sensors, communication interfaces, and the required power and signal connections.

The embedded firmware was developed to manage actuator commands, monitor the system motion through the encoder, acquire the pneumatic sensor signals at defined time intervals, and exchange commands and measurement data with a laptop. In parallel, computer software was implemented to configure the operating parameters, control the simulator, visualize the acquired signals, and support the execution of the different respiratory modes.

The overall objective was therefore to integrate the mechanical, electronic, firmware, and software subsystems into a complete and operational prototype, and to verify its ability to generate the required respiratory airflow patterns under controlled and repeatable conditions.

2 PCB Design

2.1 System Architecture

The main purpose of the electronic system is to acquire data from the sensors, process them on the main MCU, and then control the motor's velocity. The architecture is functionally divided into data acquisition/processing and in a dedicated power management section.

2.1.1 Logic and Data Acquisition

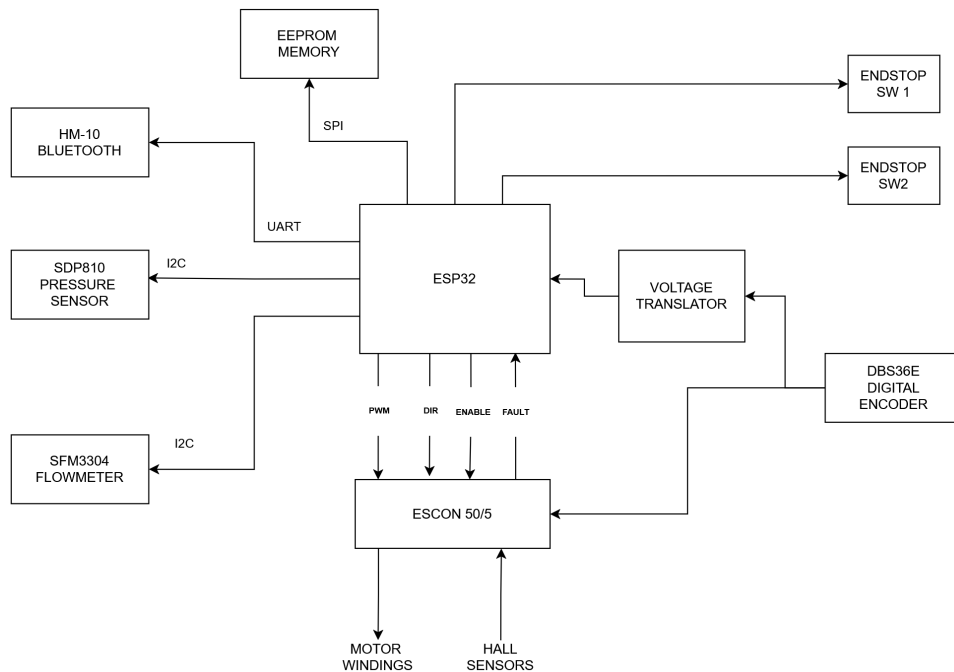


Figure 2.1: Logic architecture of the system

As shown in the system diagram (2.1), multiple peripherals interface with the ESP32 to monitor and control every aspect of the system:

- **Sensirion SDP810 Pressure Sensor**

Operating as the primary sensor in the control loop, this device senses the pressure difference generated by a custom pneumotachograph at the syringe output, estimating the output flow. It communicates with the ESP32 via an I2C bus

- **Sensirion SFM3304 Digital Flowmeter**

A secondary commercial flowmeter used to directly measure the output flow. This device is essential to calibrate the primary data source and shares the same I2C line. To ensure reliable operation in highly humid environments, an integrated anti-condensation heater can be triggered by the MCU.

- **DBS36E Digital Encoder**

Although not strictly required for the main feedback, a digital encoder tracks the position of the syringe. These data are used internally by the motor driver for positional feedback, and are used by the MCU to calculate software safety limits. A voltage translator is interposed between the encoder and the ESP32 to shift logic levels to standard 3.3V.

- **Hall Sensors**

The motor has built-in hall effect sensors that are used by ESCON to correctly drive the windings.

- **Endstop Switches**

Placed at the physical boundaries of the lead screw, two hardware switches complement the encoder to guarantee safety and to perform startup position recovery.

- **EEPROM Memory**

A dedicated non-volatile memory is included on the board to store permanent system parameters, such as PID gains, calibration curves, and software limits. This peripheral interfaces with ESP32 via a dedicated SPI bus.

- **HM10 Bluetooth Module**

The communication between Microcontroller and PC is handled via Bluetooth by an HM10 module. The device uses a UART peripheral to exchange data.

2.1.2 Motor Actuation

Motor control is handled by the ESCON 50/5, a commercial driver for DC brushless motors capable of delivering up to 5A of continuous current and 10A of peak current, providing enough torque to reach the target output flows. The driver is controlled directly by the ESP32 with three logical signals: PWM (to set speed), DIR (to set cw or ccw rotation direction) and ENABLE (to activate motor rotation). Additionally, a FAULT feedback line monitors possible motor related errors.

2.1.3 Power Management

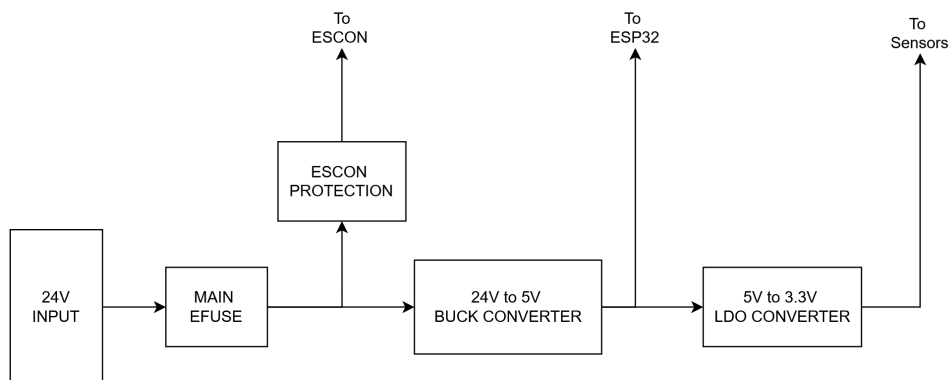


Figure 2.2: Power tree of the system

The system power tree is crucial for maintaining stability, given the relevant power requirements for motor actuation.

Because of the motor's operating voltage requirements, the system is powered by an external AC-DC converter supplying 24V. This input is safeguarded by an eFuse, which protects both the PCB and the external supply from potential overcurrent and overvoltage issues.

From this protected 24V line, power is delivered across the system:

- **Motor Supply**

The 24V line is routed to the ESCON driver. To prevent reverse current problems, a protection circuit has been designed to decouple the supply from the driver power section.

- **Logic Supply**

The system features a dual voltage logic architecture. A buck converter steps down the main 24V to a 5V rail, which primarily powers the ESP32 board and the logic part of the ESCON driver. Sequentially, an LDO regulator generates a 3.3V rail that mainly powers the sensors and peripherals such as the EEPROM memory.

2.2 Schematic Design

Schematics documents are designed to meet clarity when read, and to represent the block scheme of the system architecture. The main sections are: MCU, Power supply, ESCON 50/5, Connectors.

2.2.1 Power Section

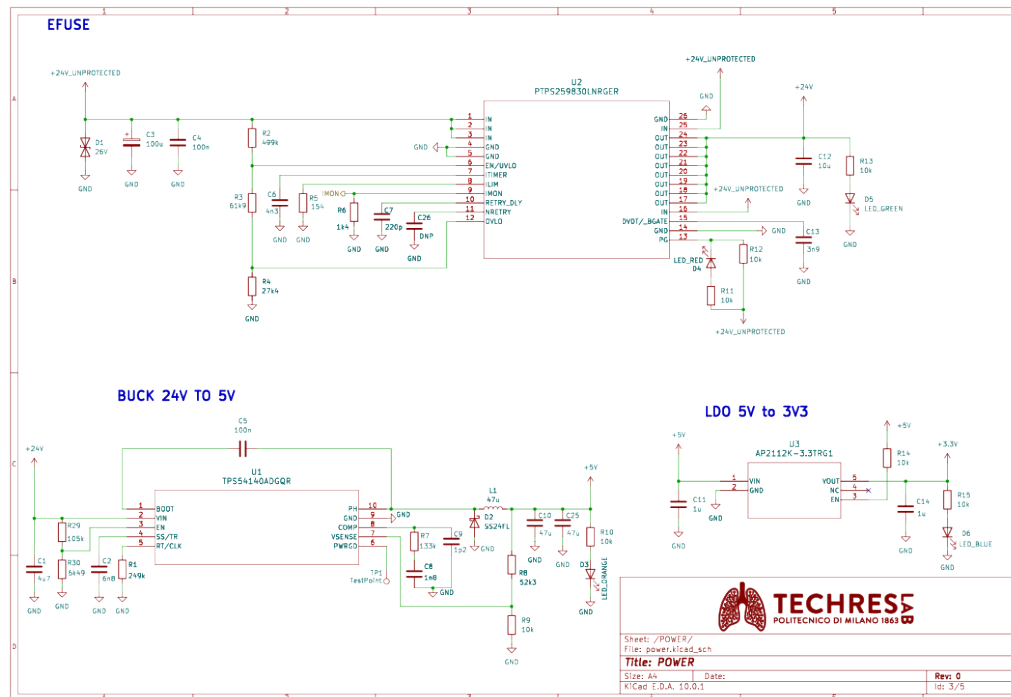


Figure 2.3: Power schematic sheet

The upper section of the schematic page (2.3) is occupied by the Efuse design. The chip used (TI TPS259830LNRGER) is sized to bring up to 10A of current , and to accept up to 26V in input. Status led have been added both on the protected 24V line and on a fault pin of the chip.

Lower section shows the components involved in the main voltage references generation: the left one is the buck converter (TI TPS54140ADGQR) and is designed to deliver up to 1.5A of stable current at 5V. The right chip is instead the LDO converter (Diodes AP2112K-3.3) that can deliver 500mA at 3.3V.

2.2.2 ESCON 50/5 and Protection

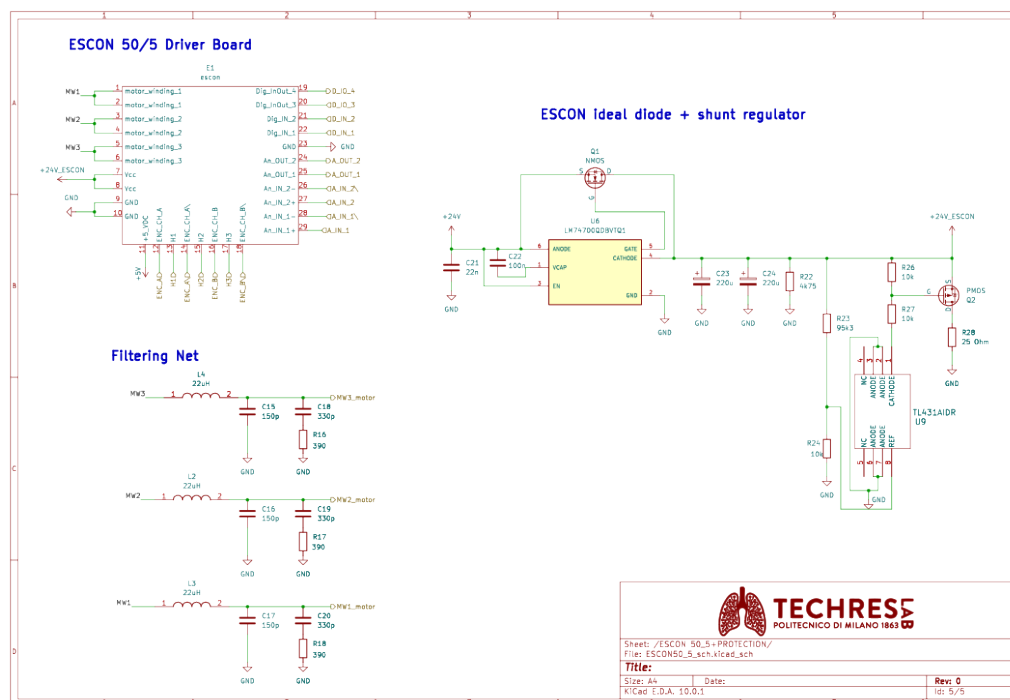


Figure 2.4: ESCON schematic sheet

To make the ESCON module work, the design features filtering networks for every motor winding, and a custom designed circuit to prevent reverse current effects due to the inductive behaviour of the motor. Focusing on the shunt circuit, the left chip (TI LM74700QDBVTQ1) drives a mosfet to act as a reverse biased diode, with zero voltage drop. This prevents the current from flowing to the rest of the 24V rail. Another component needed for the protection is the shunt regulator (TI TL431), which discharges the return current on a 25 ohm resistor when voltage increases over a threshold of 26V.

2.2.3 MCU and EEPROM

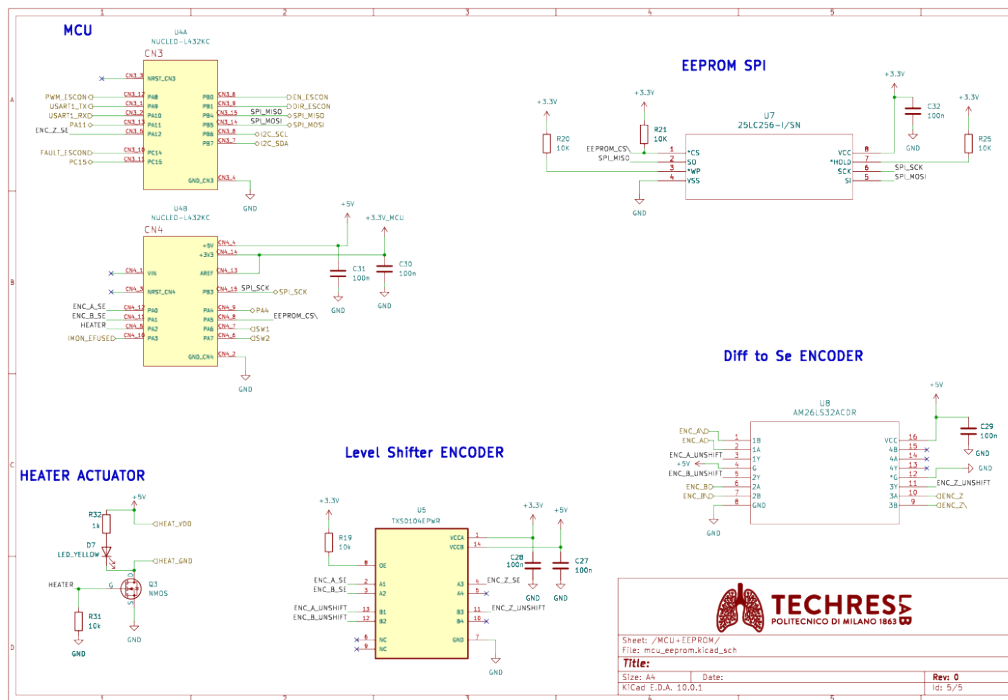


Figure 2.5: Microcontroller schematic sheet

The most important schematic part is the one regarding Microcontroller board, EEPROM and voltage translation for the encoder data. Here we should note that, in the original design, the MCU selected for calculations was an STM32 board; as will be described in the issues section, the chip has been later switched to an ESP32.

The EEPROM chip (Microchip 25LC256-I/SN) is directly connected on the MCU and features a minimal design, with only some strapping resistors and decoupling capacitors.

The data from encoder have a 2-step translation: a first device (TI AM26LS32ACDR) performs the conversion from differential to single-ended signals while the second device (TI TXS0104EPWR) shifts the logic levels from 5V of the sensor to 3.3V of the MCU.

Another important feature integrated in the project is the presence of a MOSFET switch, controlled by the Microcontroller, that activates the anti-condensation heater of the SFM3304 sensor.

SENSORS

SDBus-Digital Pressure sensor

SCL → SCL_1 J1
 +3.3V → GND J1
 SDA → SDA_1 Conn_01x04

SFM330A-xxx-D Flow Meter

HEAT_VCC → SCL J2
 GND → SCL_1 J2 Conn_01x08
 +3.3V → HEAT_GND J2

DB536/50 Encoder

ENC_BV → ENC_B J3
 ENC_BW → ENC_BW J3
 ENC_AV → ENC_A J3
 ENC_AW → ENC_AW J3
 ENC_ZV → ENC_Z J3
 ENC_ZW → ENC_ZW J3
 GND → GND J3
 +5V → +5V J3 Conn_01x08

HC05 Bluetooth

+5V → GND J4
 GND → HC_TXD J4 Conn_01x04
 HC_RXD → HC_RXD J4

EXTRA HEADER

SPI Header

+5V → 1 J8
 +3.3V → 2 J8
 SPI_MISO → 3 J8 Conn_01x06
 SPI_MOSI → 4 J8
 SPI_SS → 5 J8
 GND → 6 J8

EXTRA GPIO

PA3 → 1 J13
 PC19 → 2 J13 Conn_01x03
 PA10 → 3 J13

POWER CONN

+24V_UNPROTECTED → J9
 GND → GND J9 Conn_01x02

ESCON

MOTOR WINDINGS

Ww3_motorD → 1 J10
 Ww2_motorD → 2 J10
 Ww1_motorD → 3 J10
 Screw_Terminal_01x03

MOTOR HALL SENSORS

GND → 1 J15
 +5V → 2 J15
 H1Q → 3 J15 Conn_01x05
 H2Q → 4 J15
 H3Q → 5 J15

ESCON EXTRA PIN

ESCON_A_IN_1 → 1 J12
 ESCON_A_IN_1V → 2 J12
 ESCON_A_IN_2V → 3 J12
 ESCON_A_IN_2Q → 4 J12
 ESCON_A_OUT_1Q → 5 J12
 ESCON_A_OUT_2Q → 6 J12
 ESCON_D_IN_1Q → 7 J12
 ESCON_D_IN_1V → 8 J12
 ESCON_D_IN_2Q → 9 J12
 ESCON_D_IN_2V → 10 J12
 ESCON_D_OUT_4Q → 11 J12 Conn_01x11

SWITCHES ENDSTOP

Switch 1

+3.3V → 1 J6
 SW_1Q → 2 J6 Conn_01x03
 GND → 3 J6

Switch 2

+3.3V → 1 J6
 SW_2Q → 2 J6 Conn_01x03
 GND → 3 J6

TECHRES
POLITECNICO DI MILANO 1863

Sheet: /CONNECTORS/
File: connector_techres.sch

Title: CONNECTORS

Size: A4 Dates: Rev: 0
K/Cad E.D.A. 10.0.1 Id: 3/5

The connector section groups all the inputs and outputs of the board in a single sheet. Connections are divided in logical blocks: Sensors, switches endstops, extra headers, ESCON and motor connections, power input.

2.3.1 Stackup and Manufacturing Choices

The board features 4 layers, divided as:

- Top layer: Fast signals and smd components
- Ground Plane : A solid ground plane to guarantee signal integrity for the data
- Power plane: A layer divided in different areas that bring the right supply to chips and modules
- Bottom layer: Slower signals and other connections

10

Another important aspect is the clear separation between the logic part on the left and the right section on the right: the use of switching regulators for the supply and the field effects generated from the motor could be a problem for logical signal, so as a best practice they are kept as distant as possible.

2.3.2 Connector Placement

First of all, the connectors position ensure a fast and efficient cable management of all the peripherals. Those are divided in sections: on the upper part are located the connectors for motor windings, hall sensors, encoder and a debug port for ESCON pins. All the sensors for flow are in the right part of the board, together with the Bluetooth module and with an extra SPI header. Connector for main power input is placed in the right edge of the board.

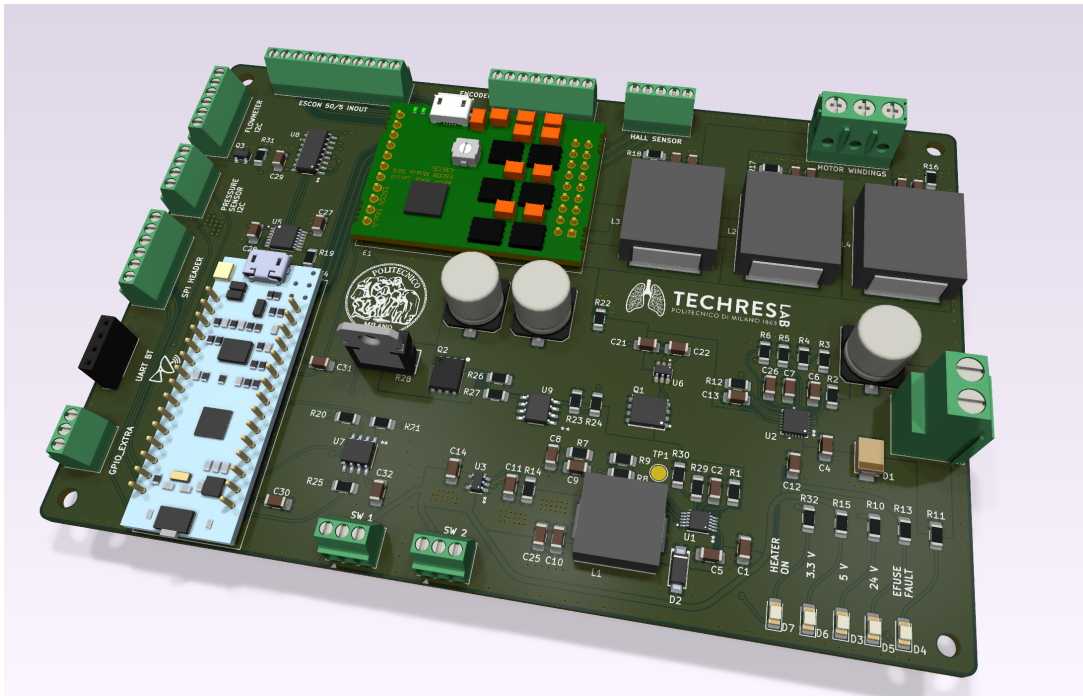


Figure 2.7: PCB Layout at version 1

3 3D Modeling and Assembly

The next challenge was to integrate all the components into a single system. The syringe, the encoder, the motor, the Sensirion flowmeter sensor, the sliding bar and the support to convert the rotary motion of the motor to the compression and retraction of the syringe were already available. However, the physical supports were old or outdated (for instance, by changing the motor to a brushless one, its support had to be changed). After taking the measurements, the Autodesk Fusion software was used to design the basic components, which were later printed in resin.

3.1 CAD Models

3.1.1 Switch Supports

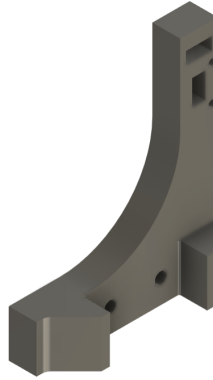


Figure 3.1: Switch support

These components were created to hold the microswitches in order to fix them in place with two screws and use them to stop the motion of the motor when activated for safety precautions. They are fixed on the sides of the main support through screws and they mark the START and END positions. The smooth curved shapes are to avoid fragility issues. Even though it is unlikely because of how it was coded, the motor can apply a high amount of force, so to avoid stability issues they do not pivot on the screw thanks to the two hooks.

3.1.2 Syringe Supports

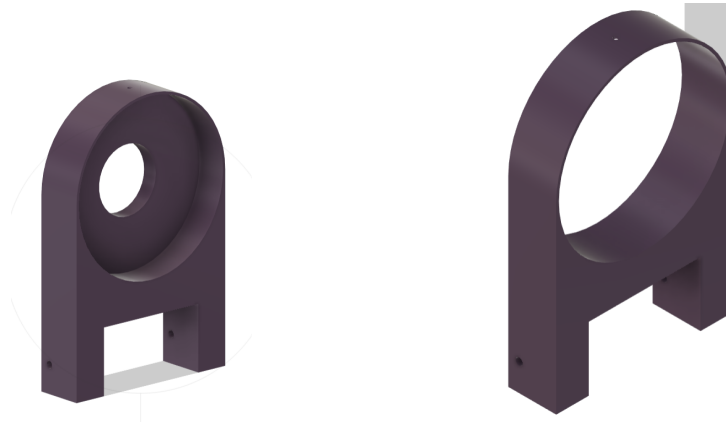


Figure 3.2: Syringe support

The syringe is supported by one component at the base and another one near the plunger. The second one (on the right) was printed in PLA because of time constraints related to resin printing. The main support instead has to withstand high forces because of the continuous pumping of the syringe. To avoid air leakage, tape was used between it and the syringe, but this led to issues related to the force required by the motor during the aspiration phase. Therefore, several holes were drilled to ease the motion.

3.1.3 Motor Support

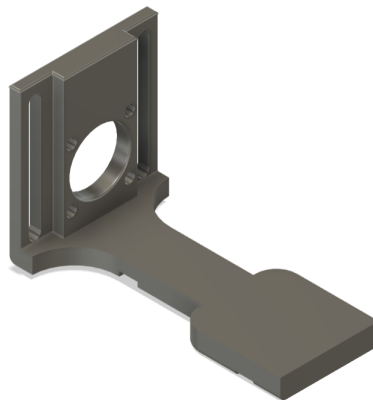


Figure 3.3: Motor support

The brushless motor is connected to the main screw, the one used to convert the motion into pumping, thanks to a 5–8 mm shaft coupling. The motor is positioned on top of the printed support, which is fixed to the main support with screws.



Figure 3.4: Shaft adapter

3.1.4 Encoder Support

The encoder is situated right behind the syringe support and connected to the main screw through a 5–8 mm shaft coupling. To improve stability, tape was used.

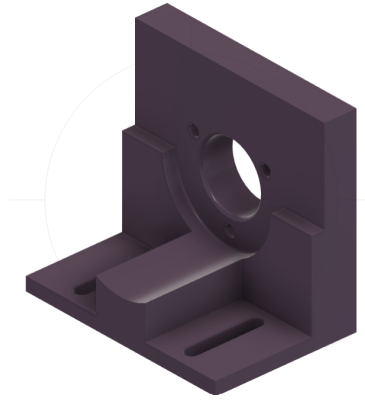


Figure 3.5: Encoder support

3.1.5 PCB Support

The main electronic control is situated behind the main support, on top of the PLA-printed backing. The PCB is fixed with two screws and the wires hang freely to the main components of the electronic board.

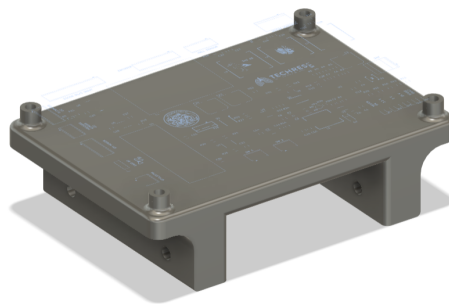


Figure 3.6: PCB Support

3.1.6 SFM3304 Adapter

The digital Sensirion flowmeter has pinouts that have to be adapted to the board to convert the sensor output into an electrically compatible signal. To this scope, a physical converter was implemented which uses the elastic shape adaptation of pogo pins to take the signal. The converter is attached to the top of the flowmeter and fixed with a latch mechanism.

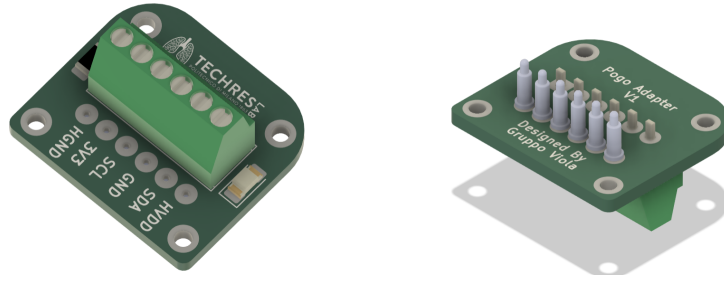


Figure 3.7: SFM3304 Adapter

3.1.7 Pneumotachograph

For feedback analysis another flowmeter is used in the project and it was designed from scratch and printed in resin. Its main features are:

- Long V-shaped mouth to adapt to the hole of the syringe.
- Two laminators of the same shape as the main resistive load (61 octagons of 0.75 mm).
- Smooth flow channel connections to resolve fragility issues.



Figure 3.8: Pneumotachograph

4 Firmware

4.1 Architecture overview

The architecture of our project is based on an ESP32 microcontroller, programmed to interface with all system peripherals. The firmware structure essentially relies on a time-driven loop, where a timer interrupt is triggered at 200 Hz. This sets a flag that is polled within the main loop to execute the critical section of the system. This approach minimizes the execution time within the Interrupt Service Routine (ISR) and mitigates potential race conditions.

Within the main loop, a Finite State Machine (FSM) orchestrates all system operations. The various states are defined in the enumeration “Modalita” and will be detailed in the following chapters.

A context wrapper, named “RespirationState”, precisely encapsulates the specific respiratory phase simulated by the device. This ensures a highly modular codebase, facilitating the seamless addition or removal of features in future iterations.

Furthermore, the system handles synchronous and asynchronous events independently. Specifically, hardware interrupts are utilized to detect external events. This enables the integration of microswitches to identify when the system reaches its mechanical limit stops.

Also, the code is very modular: Every I/O function features an abstraction function, allowing a user to easily replace them in case they need to implement the device with different sensors or actuators.

4.2 Memory Organization and Data Structures

The primary objective of this code section is to maximize structural organization and readability. To achieve this, we implemented various structures and enumerations:

Enum “Modalita”

```
1 enum Modalita { //IL CODICE RIPRODUCE UNA MACCHINA A STATI
2     MODE_LINEAR = 0,
3     MODE_SINUSOIDAL = 1,
4     MODE_CALIBRATION = 2,
5     MODE_PUSH = 3,
6     MODE_HOME = 4,
7     MODE_IDLE = 5,
8     MODE_CUSTOM_FLOW = 6, // NUOVO: riproduzione profilo flusso custom
9     MODE_DEBUG_SWING = 520,
10    MODE_DEBUG = 1000,
11    MODE_SELF_TEST = 9999
12 };
```

This serves as the state lexicon for our Finite State Machine (FSM), defining all possible operational states:

- **MODE_IDLE**: The motor is disabled, and the system awaits a command.
- **MODE_LINEAR**: Generates a setpoint for constant flow, calculated as the ratio between the target volume and the half-period. A PID controller is utilized to track the target.
- **MODE_SINUSOIDAL**: Generates a variable flow setpoint approximating a sinusoidal curve, thereby simulating a physiological breathing pattern. This is constrained by the maximum volume limit, with active PID control.
- **MODE_CUSTOM_FLOW**: Interpolates an array of instantaneous flow setpoints pre-loaded into memory, enabling the reproduction of customized curves via PID control.
- **MODE_CALIBRATION**: Executes motion in discrete fractional increments to verify volumetric accuracy.
- **MODE_PUSH**: Executes manual movement at a constant speed in a specified direction, operating in an open-loop configuration.
- **MODE_HOME**: Executes a slow, constant movement towards the home (rest) position.
- **MODE_DEBUG_SWING**: Mechanical stress-testing mode. The motor operates at a constant PWM duty cycle in a single direction until software limits are triggered.
- **MODE_DEBUG**: System diagnostic and debugging mode.
- **MODE_SELF_TEST**: Initialization routine designed to calibrate and store the end-stop position values.

These state definitions ensure mutually exclusive execution.

struct "ControllerConfig"

```

1 struct ControllerConfig { //STRUTTURE DATI CHE INCLUDONO PARAMETRI DI CONTROLLO
   , STATO E VALORI LETTI DI VARI SETTORI DEL CODICE
2     float kp = 0.05f;
3     float ki = 0.005f;
4     float kd = 0.0f;
5     float kff = 0.10f;
6     float kff_diff[2] = { 0.0f, 0.0f };
7     float kff_quad = 0.02f;
8     float resistance = 1.0f;
9     float syringe_volume_L = 1.0f;
10 } config;

```

Contains all PID tuning parameters and system mechanical constants. This structure facilitates the seamless uploading and saving of calibration profiles to the EEPROM.

Struct "RespirationState"

```

1 struct RespirationState {

```

```

2   Modalita mode = MODE_IDLE;
3   TipoSensore sensor = NESSUNO;
4   float target_volume_L = 0.0f;
5   float semiperiod_sec = 0.0f;
6   float accumulated_vol_L = 0.0f;
7   float current_flow = 0.0f;
8   float pressure = 0.0f;
9   bool is_inspiration = true;
10  unsigned long phase_start_ms = 0;
11
12  int tick = 0;
13  float integral = 0.0f;
14  float prev_error = 0.0f;
15  float last_output_error = 0.0f;
16  unsigned long prev_pid_time = 0;
17 } breath;

```

Serving as the context wrapper, this structure represents the real-time state of the system throughout its operation, acting as a dynamic snapshot. It encapsulates:

- Target kinematics (e.g., volume and time).
- Sensor data (e.g., instantaneous flow, pressure, and computed accumulated volume).
- Current PID variables and motor status.

This architecture significantly enhances state observability and allows for a safe, comprehensive reset of the system context when necessary.

Struct “MotorState”

```

1 struct MotorState {
2     float current_pwm_norm = 0.0f;
3     int current_pwm_raw = 0;
4     bool direction = false;
5     bool enabled = false;
6     bool flag_invert_req = false;
7     bool flag_near_end = false;
8     bool flag_too_near = false;
9
10    float swing_pwm = 0.0f;
11    bool swing_dir = true;
12 } motor;

```

Contains the current PWM duty cycle applied to the motor, along with a set of flags dedicated to handling boundary conditions when the carriage approaches critical dynamic limits. Its primary purpose is to manage motor pin configurations, providing a hardware abstraction layer for the system.

Struct “HardwareVolatiles”

```
1 struct HardwareVolatiles {
2     volatile bool sample_200hz = false;
3     volatile bool microswitch_clash = false;
4     volatile long encoder_pos = 0;
5     volatile long max_encoder_steps = 0;
6     volatile unsigned long last_irq_ms = 0;
7 } hw;
```

Groups all volatile variables within the system, including timer flags, microswitch interrupt flags, and encoder step counts. These are strictly declared as `volatile` to force the compiler to fetch their current values directly from RAM upon every access, as they can be modified asynchronously by hardware interrupts.

Struct “CustomFlowState”

```
1 struct CustomFlowState {
2     float flow_wp[CFLOW_MAX_WAYPOINTS];    // [L/s]  (positivo = inspirazione,
3     //      negativo = espirazione)
4     uint32_t time_wp[CFLOW_MAX_WAYPOINTS]; // [ms]   dalla partenza
5     uint16_t num_wp = 0;
6     uint16_t current_wp = 0;    // indice del waypoint corrente (precedente)
7     unsigned long start_ms = 0; // millis() al via
8     bool armed = false;
9 } cflow;
```

The custom flow mode necessitates specific optimization due to its high computational complexity. This structure encapsulates all the data required to map a custom breathing pattern. Specifically, it contains arrays storing all pre-calculated waypoints and their corresponding timestamps.

4.3 Interrupt Service Routines

The real-time architecture of our system requires a strict decoupling of synchronous (time-driven) and asynchronous (event-driven) events. Consequently, we implemented an interrupt architecture designed to handle and categorize all incoming events. It is crucial to note that the Interrupt Service Routines (ISRs) refrain from executing computationally intensive algorithms; instead, they are strictly limited to setting boolean flags. This defers the actual data processing to the main loop, thereby preventing bottleneck conditions and ensuring overall firmware stability.

4.3.1 Synchronous events

All these events are governed by a hardware timer configured to generate an interrupt every 5 ms (200 Hz).

```
1 timer200Hz = timerBegin(1000000);    // Risoluzione a 1 MHz (1
    tick = 1 us)
```

```

2 timerAttachInterrupt(timer200Hz, &onTimer200Hz); // Aggancio della ISR
3 timerAlarm(timer200Hz, 5000, true, 0);           // Allarme sincrono ogni
           5000 us (5 ms -> 200 Hz) con ricarica automatico (true)
4
5 void IRAM_ATTR onTimer200Hz() {
6     hw.sample_200hz = true;
7 }

```

The routines executed within this temporal constraint include:

- **Sensor reading:** The I2C bus is polled at exactly 200 Hz to acquire uniformly sampled data.

```

1 float valoreletto = NAN;
2     breath.pressure = NAN;
3
4     if (breath.sensor == SDP810) {
5         valoreletto = leggi_SDP810();
6         if (!isnan(valoreletto)) {
7             breath.pressure = valoreletto;
8             breath.current_flow = valoreletto / config.resistance;
9         }
10    } else if (breath.sensor == SFM3300) {
11        valoreletto = leggi_SFM3300();
12        if (!isnan(valoreletto)) breath.current_flow = valoreletto;
13    } else {
14        breath.current_flow = flow_from_encoder;
15        valoreletto = breath.current_flow;
16    }

```

- **Setpoint generation and PID:** In custom flow mode, the system tracks the elapsed time from the start of the pattern. Based on the specified period and the adjacent waypoints, it computes the instantaneous target value via interpolation. Furthermore, it evaluates the terms required for closed-loop control. Notably, our controller extends the traditional Proportional-Integral-Derivative (PID) algorithm by incorporating a feed-forward term. This predictive component actively compensates for the pressure drops within the pneumatic system, modeled according to Poiseuille's law.
- **Telemetry:** This block includes a prescaler counter. Every ten timer ticks, it sets a transmission flag to schedule Bluetooth communication. This downsampling guarantees data transmission at exactly 20 Hz, which is well within the bandwidth limitations of the HM-10 BLE module.

```

1 // BLE Telemetry Downsampling (200 Hz to 20 Hz)
2 static int contatore_ble = 0;
3 contatore_ble++;
4
5 if (contatore_ble >= 10) { // Condition met every 10 timer ticks (50 ms)
6     contatore_ble = 0;
7     char msg_buf[64];
8
9     // Format the data payload
10    snprintf(msg_buf, sizeof(msg_buf), "SNSR %.3f %ld %d %.3f %.3f \r\n",

```

```

11         breath.current_flow, hw.encoder_pos, motor.current_pwm_raw,
12         breath.last_output_error, breath.pressure);
13
14     // Asynchronous transmission over BLE characteristic
15     if (deviceConnected && pTxCharacteristic != nullptr) {
16         pTxCharacteristic->setValue((uint8_t*)msg_buf, strlen(msg_buf));
17         pTxCharacteristic->notify();
18     }
19 }

```

4.3.2 Asynchronous events

The primary asynchronous events in this architecture are triggered by the microswitches—acting as the fundamental hardware safety mechanism of the machine—and the two channels of the rotary encoder:

- **End-stops:** Two limit switches are positioned at the extremities of the lead screw to be physically actuated by the carriage. Without this intervention, the carriage would collide with the mechanical boundaries, potentially inducing a motor stall condition, leading to excessive current draw and thermal overload.

```

1 void IRAM_ATTR onEmergencyStop() {
2     unsigned long tempo_attuale = millis();
3
4     // Software Debounce: Ignora i trigger successivi al primo per 3000 ms
5     if (tempo_attuale - hw.last_irq_ms > 3000 &&
6         (digitalRead(PIN_LIMIT_SW1) == LOW || digitalRead(PIN_LIMIT_SW2) ==
9             LOW)) {
10
11         hw.microswitch_clash = true; // Flag asincrono letto dal loop
12         principale
13         hw.last_irq_ms = tempo_attuale;
14     }
15 }

```

- **Encoder channels:** The encoder generates pulses continuously as the motor drives the carriage. Every rising or falling edge triggers a hardware interrupt, allowing the system to accurately track and update the absolute position of the carriage in real-time. We don't use the Z channel as it provides less resolution. The first peripheral inputs necessitate the implementation of a digital debounce filter to prevent false triggering and interrupt storms. Meanwhile, the encoder uses a look-up table to decode the quadrature.

```

1 void IRAM_ATTR onEncoderISR() {
2     // Quadrature Decoding Lookup Table: mappa le transizioni di stato in
3     incrementi (+1, -1, o 0)
4     static int8_t lookup_table[] = { 0, -1, 1, 0, 1, 0, 0, -1, -1, 0, 0, 1,
5         0, 1, -1, 0 };
6     static uint8_t enc_val = 0;
7
8     // Bare-Metal Register Access: Lettura simultanea dei pin GPIO a
9     bassissimo livello
10    uint32_t reg = REG_READ(GPIO_IN1_REG);

```

```

8  uint8_t a = (reg >> 2) & 1; // Estrae lo stato del PIN_ENC_A (GPIO 34)
9  uint8_t b = (reg >> 3) & 1; // Estrae lo stato del PIN_ENC_B (GPIO 35)
10
11  // Costruzione dell'indice binario e aggiornamento della variabile
    volatile
12  enc_val = enc_val << 2;
13  enc_val = enc_val | ((a << 1) | b);
14  hw.encoder_pos += lookup_table[enc_val & 0b1111];
15 }

```

4.4 Bluetooth communication

The system requires bidirectional PC communication for supervisory control, telemetry logging, and parameter configuration. To achieve this, we integrated an HM-10 module, which acts as a wireless UART bridge over Bluetooth Low Energy (BLE). The communication interface operates in an event-driven idle state, utilizing interrupt callbacks triggered only upon the reception of incoming data payloads.

```

1  class MyCallbacks : public BLECharacteristicCallbacks {
2  void onWrite(BLECharacteristic* pCharacteristic) override {
3      String rxValue = pCharacteristic->getValue();
4      if (rxValue.length() > 0) {
5          Serial.print(rxValue);
6          for (int i = 0; i < (int)rxValue.length(); i++) {
7              char c = rxValue[i];
8              if (c == '\n' || c == '\r') {
9                  if (ble_rx_index > 0) {
10                     parser();
11                     ble_rx_index = 0;
12                     ble_rx_buffer[0] = '\0';
13                 }
14             } else {
15                 if (ble_rx_index < sizeof(ble_rx_buffer) - 1) {
16                     ble_rx_buffer[ble_rx_index++] = c;
17                     ble_rx_buffer[ble_rx_index] = '\0';
18                 }
19             }
20         }
21     }
22 }
23 };

```

4.4.1 Data Reception

Implementing a blocking wait to receive and interpret an entire string sequentially would severely compromise the real-time performance. Such an approach would induce control loop jitter, halting system execution and unnecessarily increasing mechanical stress on the motor. To mitigate this, incoming data bytes are asynchronously appended to a static buffer. CPU execution is diverted only briefly to copy the payload into RAM, ensuring the motor control loop remains undisturbed. This non-blocking buffering process continues iteratively until a predetermined termination character is detected.

4.4.2 Command Parser

Upon detection of the termination character, a dedicated `parser()` routine is invoked to decode the buffered payload and execute the corresponding command.

The parser supports the following instruction set:

- Mode 0 <volume> <time>: Activates linear flow mode.
- Mode 1 <volume> <time>: Activates sinusoidal flow mode.
- CFLOW <flow1> <time1> <flow2> <time2> ...: Activates custom interpolated flow mode.
- Mode 7 <direction> <speed>: Activates manual open-loop carriage displacement.
- Mode 4 <direction>: Initiates the homing sequence.
- STOP: Immediately halts all actuation.

Additionally, it accepts configuration commands for controller tuning:

- PID <Kp> <Ki> <Kd>: Overwrites the proportional, integral, and derivative gains.
- KFF <value>: Sets the linear feed-forward base gain.
- KFFDiff <direction0> <direction1>: Configures the asymmetric feed-forward offset.
- KFFQuad <value>: Sets the quadratic feed-forward coefficient.
- R <value>: Defines the pneumatic/hydraulic resistance.
- Volume <value>: Defines the total volumetric capacity of the system.

Furthermore, the interface supports calibration and non-volatile memory operations:

- Mode 9999: Triggers the self-calibration routine to map hardware limit switch boundaries.
- Mode 2 <volume> <time> <steps>: Initiates resistance curve calibration, streaming output data in .csv format for subsequent plotting.
- FWV:<string>: Commits the current controller configuration to EEPROM for persistent storage.

4.4.3 Data Transmission

```
1 void send_BLE(const String& messaggio) {
2   if (deviceConnected && pTxCharacteristic != nullptr) {
3     String payload = messaggio + "\r\n";
4     pTxCharacteristic->setValue(payload.c_str());
5     pTxCharacteristic->notify();
6   }
7 }
```

As previously mentioned, to accommodate the HM-10 module's bandwidth limitations, telemetry data transmission is downsampled to 20 Hz. This strategy is essential to prevent transmit buffer overflow and client-side saturation.

4.4.4 GUI synchronization

The microcontroller acts as the single source of truth for the system's configuration. Therefore, upon establishing a connection with the host PC, the firmware initiates a brief delay before transmitting the persistent PID parameters stored in the EEPROM to the client. This initialization sequence ensures full synchronization between the host interface and the current hardware state.

4.5 PID Architecture

As the system is strongly non linear, we need a PID that's able to track the flow even in challenging scenarios. Due to that, the PID features, apart from the classic PID parameters, a second degree, asymmetric, feed forward polynomial to improve speed, accuracy and reduce ringing

```
1  {
2      float error = setpoint - flow_abs;
3
4      int dir_idx = breath.is_inspiration ? 0 : 1;
5      float base_ff = config.kff + config.kff_diff[dir_idx];
6      float ff_term = (setpoint * base_ff) + (config.kff_quad * setpoint *
          setpoint);
7
8      float raw_derivative = (error - breath.prev_error) / dt;
9      static float filtered_derivative = 0.0f;
10     filtered_derivative = (0.2f * raw_derivative) + (0.8f *
          filtered_derivative);
11
12     float p_term = config.kp * error;
13     float d_term = config.kd * filtered_derivative;
14     float tentative_output = ff_term + p_term + (config.ki * breath.
          integral) + d_term;
15
16     const float OUT_MAX = MOTOR_MAX_PWM_NORM;
17     const float OUT_MIN = 0.0f;
18
19     bool hit_upper = (tentative_output >= OUT_MAX);
20     bool hit_lower = (tentative_output <= OUT_MIN);
21
22     if (!hit_upper && !hit_lower) {
23         breath.integral += error * dt;
24     } else if (hit_upper && error < 0.0f) {
25         breath.integral += error * dt;
26     } else if (hit_lower && error > 0.0f) {
27         breath.integral += error * dt;
28     }
29
30     float final_output = ff_term + p_term + (config.ki * breath.integral) +
          d_term;
31     if (final_output > OUT_MAX) final_output = OUT_MAX;
32     if (final_output < OUT_MIN) final_output = OUT_MIN;
33
34     breath.last_output_error = final_output;
```

```
35     breath.prev_error = error;
36
37     setMotor(final_output, !breath.is_inspiration, true);
38 }
39 }
```

5 GUI

5.1 Main goal

To govern the system, we developed a dedicated host software application designed to establish bidirectional communication. The core component is a Graphical User Interface (GUI) tailored for transmitting commands, monitoring system telemetry, and rendering real-time data visualizations.

The application is built upon the PyQt6 framework, leveraging the Qt toolkit to ensure a highly optimized and professional-grade user experience. Furthermore, the **bleak** library is integrated to manage Bluetooth Low Energy (BLE) connectivity, guaranteeing cross-platform compatibility across various operating systems.

Finally, data plotting is handled via **pyqtgraph**. This library was specifically selected over **matplotlib** due to its superior optimization and rendering performance in high-refresh-rate, real-time applications.

5.2 Communication management

PyQt6 operates on a synchronous event loop, meaning that any blocking I/O or computationally expensive operation will render the Graphical User Interface (GUI) unresponsive. Conversely, the 'bleak' Bluetooth library is natively asynchronous, creating an inherent architectural incompatibility out of the box.

```
1 if __name__ == "__main__":
2     app = QApplication(sys.argv)
3
4     # 1. Creazione dell'event loop ibrido tramite qasync
5     loop = qasync.QEventLoop(app)
6     asyncio.set_event_loop(loop)
7
8     window = HM10App()
9     window.show()
10    QTimer.singleShot(0, window.btn_scan.click)
11
12    # 2. Avvio del loop concorrente (gestisce sia UI che BLE)
13    with loop:
14        loop.run_forever()
```

To resolve this, we integrated the **qasync** library. This module seamlessly merges the standard Python **asyncio** event loop with the PyQt6 **QEventLoop**, enabling time-consuming BLE routines to execute concurrently. Consequently, the application can manage asynchronous network operations within a single-threaded architecture while maintaining full GUI responsiveness.

```

1 @asyncSlot()
2     async def connect_device(self):
3         # ... logica asincrona non bloccante per connettersi al BLE ...
4         self.client = BleakClient(address)
5         await self.client.connect()
6         # ...

```

Regarding data reception on the client side, incoming payloads are similarly buffered. The data fragments are iteratively appended to an array until a termination character is detected. This mechanism ensures the complete and accurate reconstruction of the message, guaranteeing data integrity before the payload is forwarded to the command parser.

```

1 def _rx_handler(self, sender, data: bytes):
2     self.rx_buffer += data
3     while b"\n" in self.rx_buffer:
4         line, self.rx_buffer = self.rx_buffer.split(b"\n", 1)
5         clean = line.replace(b"\r", b"").decode("utf-8", errors="ignore").
6             strip()
7         asyncio.get_event_loop().call_soon_threadsafe(self._process_line,
8             clean)

```

5.3 State synchronization and input validation

Upon establishing a connection with the host PC, the microcontroller immediately transmits the PID parameters stored in the EEPROM alongside the firmware version, encapsulated within a unique data string. This procedure ensures the full synchronization of the host's real-time system state with the device's persistent configuration.

The `_parse_and_populate_rom` function receives this payload, parses the string, and populates the corresponding internal variables with the extracted values.

```

1 def _parse_and_populate_rom(self, line: str):
2     clean = line[4:].strip()
3     tokens = clean.split()
4
5     if len(tokens) < 17 or tokens[1] != "PIDSDP" or tokens[9] != "PIDENC":
6         self.txt_debug.append(f"[ROM ERR] Formato EEPROM incompatibile: {
7             line}")
8         return
9
10    try:
11        self.rom_fw.setText(tokens[0])
12        keys = ["kff", "kffd0", "kffd1", "kff_quad", "kp", "ki", "kd"]
13
14        for i, k in enumerate(keys):
15            self.rom_sdp[k].setText(tokens[2 + i])
16            self.rom_enc[k].setText(tokens[10 + i])

```

```

16         self.txt_debug.append("[ROM] Parametri EEPROM uC caricati nella GUI
17             con successo.")
18     except Exception as e:
19         self.txt_debug.append(f"[ROM ERR] Errore popolamento GUI: {e}")

```

Furthermore, the GUI serves as a robust validation layer, permitting the upload of custom flow waypoints only if the dataset strictly adheres to the following constraints:

- A minimum of two waypoints must be defined.
- The initial timestamp must be strictly equal to zero.
- The timestamp values must increase monotonically.

If any of these conditions are violated, the application rejects the input and generates an error prompt. Following a successful validation, the system transmits the command payload previously detailed in Chapter 4.4.

5.4 Plot Rendering

This rendering process represents a potential performance bottleneck for the GUI; direct, synchronous updates could induce severe latency and frame drops due to the bandwidth constraints of the communication link. To mitigate this, we decoupled the data acquisition and rendering tasks:

Initially, incoming telemetry data is asynchronously appended to a buffer array upon arrival.

```

1  t = time.time() - self.start_time
2      self.t_data.append(t)
3      self.flow_data.append(flow)
4      self.pos_data.append(pos)
5      self.pwm_data.append(pwm)
6      self.pid_err_data.append(pid_out)
7      self.pressure_data.append(pressure)
8      self._plot_dirty = True

```

Concurrently, a dedicated software timer triggers every 33 ms (achieving an effective refresh rate of approximately 30 Hz) to poll for buffer updates. If new data is detected, the plotting routines are executed. This architectural decoupling ensures optimal rendering fluidity and prevents user interface blocking.

Finally, the visualized data is displayed using a 10-second rolling window to provide a continuous temporal overview of the system's dynamics.

```

1  def _refresh_plots(self):
2      if not self._plot_dirty or not self.t_data:

```

```

3         return
4     self._plot_dirty = False
5
6     t = list(self.t_data)
7
8     if not self.is_calibrating:
9         cutoff = t[-1] - ROLLING_WINDOW_S
10        idx = next((i for i, v in enumerate(t) if v >= cutoff), 0)
11        t = t[idx:]
12        flow = list(self.flow_data)[idx:]
13        pos = list(self.pos_data)[idx:]
14        pwm = list(self.pwm_data)[idx:]
15        pid = list(self.pid_err_data)[idx:]
16        pre = list(self.pressure_data)[idx:]
17    else:
18        flow = list(self.flow_data)
19        pos = list(self.pos_data)
20        pwm = list(self.pwm_data)
21        pid = list(self.pid_err_data)
22        pre = list(self.pressure_data)
23
24    self.line_flow.setData(t, flow)
25    self.line_pressure.setData(t, pre)
26    self.line_pos.setData(t, pos)
27    self.line_pwm.setData(t, pwm)
28    self.line_pid_err.setData(t, pid)

```

5.5 Data logging abstraction

Since the ESP32 lacks an onboard filesystem (such as an SD card module) to store large datasets, the PC application acts as the primary data logger during the `MODE_CALIBRATION` sequence.

5.5.1 File I/O Abstraction

To handle data recording systematically, a dedicated `CalibrationLogger` class was implemented.

```

1 class CalibrationLogger:
2     def __init__(self, ref_volume: float, resistance: float):
3         self.ref_volume = ref_volume
4         self.resistance = resistance
5
6         ts = int(time.time())
7         self.path = f"calibration_{ts}.csv"
8         self._file = open(self.path, mode="w", newline="", encoding="utf-8")
9         self._w = csv.writer(self._file)
10
11        self._w.writerow(["# Reference Volume (L)", ref_volume])
12        self._w.writerow(["# Resistance", resistance])
13        self._w.writerow(["# Started", time.strftime("%Y-%m-%dT%H
        :%M:%S")])

```

```

14         self._w.writerow([])
15         self._file.flush()
16
17         self.current_step      : int | None = None
18         self.step_start_time   : float | None = None
19         self._header_written   = False
20
21     def begin_step(self, step_number: int):
22         self.current_step      = step_number
23         self.step_start_time   = time.time()
24
25         self._w.writerow([])
26         self._w.writerow([f"# --- STEP {step_number} ---"])
27
28         if not self._header_written:
29             self._w.writerow(["Step", "Timestamp_s", "RelTime_s", "Pressure_Pa"
30                               ])
31             self._header_written = True
32
33         self._file.flush()
34
35     def log_sample(self, pressure: float):
36         if self.current_step is None or self.step_start_time is None:
37             return
38         abs_t = time.time()
39         rel_t = abs_t - self.step_start_time
40         self._w.writerow([
41             self.current_step,
42             f"{abs_t:.4f}",
43             f"{rel_t:.4f}",
44             f"{pressure:.5f}",
45         ])
46
47     def end_step(self):
48         self._file.flush()
49
50     def close(self) -> str:
51         self._file.flush()
52         self._file.close()
53         return self.path

```

Upon initialization, it generates a unique .csv file appended with a UNIX timestamp to prevent the accidental overwriting of previous tests. It immediately writes a metadata header containing the user-defined Reference Volume and pneumatic Resistance, ensuring each dataset is self-contained for future offline analysis.

5.5.2 Event-Driven Synchronization

The logging process is not continuous but strictly synchronized with the microcontroller's mechanical steps via string token handshakes. When the firmware completes a movement and stabilizes, it transmits a READY token followed by the step index. The GUI intercepts this event, resets a local relative timer, and begins logging the high-resolution pressure samples. Conversely, a TURN token signals the end of the mechanical step, halting the data acquisition

for that block.

5.5.3 Data Integrity

To safeguard against potential data loss caused by unexpected BLE disconnections or software crashes, the logger explicitly calls the `flush()` method at key transitions. This forces the host operating system to immediately write the memory buffers to the physical storage drive, ensuring that all acquired calibration data is safely preserved.

6 Main Challenges and Solutions

Throughout the realization of the project, among the main sections (such as the code, the 3D design and printing, the PCB board, and the assembly part), some problems were encountered. These issues were challenging and required problem-solving and analysis to find a solution.

6.1 Software Issues

6.1.1 Outdated Bluetooth Module

In 2026, the drivers for the HC05 module used for Bluetooth communication stopped working correctly. After debugging, the best solution was to change the module. At first an ESP32 microcontroller was used only for communication purposes; then the updated HM10 communication device solved the problem.

6.1.2 UART Communication Complications

The STM microcontroller uses UART to communicate with the ST-Link and the GUI. It took a while to understand that the protocol stopping working meant the microcontroller board had failed. At first it was replaced with another one, but to make the code more compact and, especially, to simplify connectivity with the other devices, the STM was then substituted with an ESP32.

6.2 Wrong Measurements for CAD Models

After realizing that the printed components were offset, too big or simply did not fit, it was better not to print them again because of time constraints. The reason the components were crooked was due to the limitations of the resin printer and inaccurate measurements. So it was decided to try to fix them through drilling and filing: sandpaper was used to make what could not fit slimmer, while a drill was used to make holes for the screws since they were offset. After the testing phase, the solution was to design them again with more accurate measurements, this time with extra features:

- A 0.1 mm margin every 10 mm; this helped the components fit better and almost perfectly.
- Smoother angles; as discussed for the Switch Support, not having sharp edges avoided fragility issues.

It is to be noticed the presence of a perfectly fixed screw between the encoder and syringe support: this is to avoid the latter moving too much because of the forces applied by the motor.

6.3 PCB Issues and Fixes

6.3.1 Main PCB Issues

During the use and testing phase of the original PCB design, some errors and critical issues arose:

- **Change of MCU**

As previously mentioned in PCB section, during the development the Microcontroller board has been updated from an STM32 to an ESP32 board, with the consequent need to adapt pinout for the module.

- **ESCON Protection circuit**

The ESCON reverse current protection circuit delivered some critical issues in the first instance, by sending the Efuse in protection mode at startup. This prevented the whole board from working and needed a bypass of the circuit.

- **I2C pull-up resistors**

By a misunderstanding in the design phase, the I2C line was left without pull-up resistors, making the communication with sensors impossible.

6.3.2 Second Review and Design Improvements

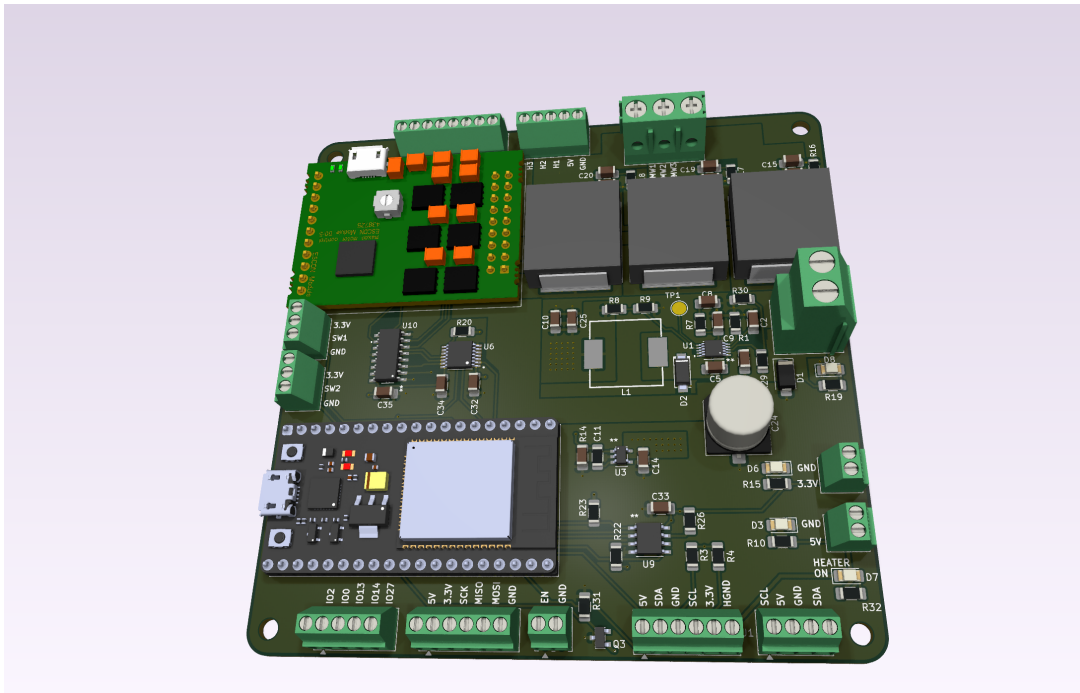


Figure 6.1: Second PCB Revision

Due to the multiple issues related to the original design, a second version of the PCB has been created. Here are the main changes applied to the project:

- **Simplified Power Supply**

During testing, the protection circuitry was found to be unnecessarily complex for the

intended application. Therefore, to reduce cost and simplify the design, both the eFuse and the malfunctioning ESCON protection circuit were removed from the second PCB revision.

- **ESP32 Integration**

As described before, the new board features the pinout for ESP32 module, leaving behind the legacy STM32 compatibility.

- **Compact design**

Due to the smaller Bill of Materials, also the size of the PCB has been reduced to meet both a better usability and a lower production cost.

- **Better Silkscreen**

To improve the user experience, all the pins of connectors now have their signal printed on the top silkscreen (in the older version this feature was present only on the bottom side, and due to mechanical mount this was not easily reachable).

- **Other minor fixes**

Some other minor issues have been addressed and corrected, such as the missing I2C pullup resistor and some wrong connected pins.

7 Testing Phase

During the final phase of the project, calibration and functional tests were performed to evaluate the operation of the integrated system and identify the remaining mechanical and control limitations.

7.1 Calibrations

The first test focused on the calibration of the resin-printed pneumotachograph using a 3 L reference syringe. By displacing a known volume over a controlled time interval, the corresponding reference flow rate was estimated. The differential-pressure measurements were then associated with the reference flow values to determine the pressure–flow characteristic and the effective pneumatic resistance of the device.

Initially, the calibration was performed manually by observing and analysing the acquired curves. The procedure was subsequently automated through dedicated software, which stored the measurements in CSV files and processed them to estimate the pressure–flow relationship and the corresponding resistance curve. The experimental results showed a nonlinear dependence of the apparent pneumatic resistance on the pressure difference, with the estimated resistance increasing as the pressure difference increased.

7.2 System Validation

Following manufacturing and assembly, some of the 3D-printed components required manual adjustments, including filing and drilling, to ensure proper alignment and integration. During the initial motor tests, the mechanical loads damaged some of the less robust supports. These components were subsequently redesigned and manufactured with increased structural strength. Further tests confirmed the improved mechanical stability of the revised solution.

Final functional tests were performed on the implemented linear, sinusoidal, and custom-flow modes using the PID-based control strategy. During the initial trials, the actuator experienced increased mechanical resistance in the aspiration phase. This issue was reduced by modifying the syringe support to improve air circulation behind the syringe. Following this correction, the system successfully executed the implemented airflow-generation modes during the performed tests. However, the tests were primarily functional, and a complete quantitative comparison between the target and measured profiles remains necessary.

8 Conclusions & Future Work

8.1 Conclusions

Project Violet led to the development of a functional adult breathing simulator based on the motor-driven displacement of a syringe plunger. Starting from a partially assembled and outdated prototype, the mechanical, electronic, firmware, and software subsystems were progressively redesigned and integrated into a single mechatronic platform.

From a mechanical perspective, new supports and adapters were designed and manufactured for the brushless motor, encoder, syringe, limit switches, electronic board, and pneumatic sensors. Several initial components presented dimensional inaccuracies or insufficient mechanical strength and therefore required manual correction or complete redesign. The final modifications improved component alignment and structural stability, allowing the system to withstand the forces generated during repeated syringe actuation.

The electronic system was developed to integrate the motor driver, flow and pressure sensors, encoder, end-stop switches, communication interfaces, and power-management circuitry. During development, the original STM32-based architecture was replaced by an ESP32-based solution to simplify peripheral integration and communication. The first PCB revision revealed several critical issues, including missing I²C pull-up resistors, incompatibility with the new microcontroller, and problems related to the ESCON protection circuitry. These limitations were temporarily mitigated through corrective modifications and hardware bypasses applied to the first PCB, enabling functional testing of the prototype. Based on the issues identified during this phase, a second, simplified PCB revision was subsequently designed, although it was not fabricated or experimentally validated within the project timeframe.

The firmware was organized around a time-driven architecture and a finite-state machine supporting multiple operating, calibration, safety, and diagnostic modes. It coordinated sensor acquisition, setpoint generation, PID-based motor control, position monitoring, and telemetry through synchronous and asynchronous events.

The pneumatic measurement system included a commercial flowmeter and a custom resin-printed pneumotachograph. The latter was calibrated by applying known volume displacements and relating the measured differential pressure to the corresponding airflow. The calibration procedure was progressively automated, allowing the acquired data to be stored and processed to estimate the pressure-flow relationship of the device.

Final functional tests showed that the simulator was able to execute the implemented airflow waveforms. Initial difficulties during the syringe aspiration phase were reduced by modifying the syringe support to improve air circulation. Following this correction and the reinforcement of the most mechanically stressed components, the system operated successfully during the performed tests.

Overall, the main project objective was achieved: the mechanical, electronic, firmware, and

software components were integrated into an operational prototype capable of executing the implemented respiratory airflow-generation modes. Nevertheless, the tests performed mainly demonstrated the functional feasibility of the system. A more extensive quantitative characterization is still required to determine its accuracy, repeatability, dynamic performance, and operating limits.

8.2 Future Work

Future development should first focus on a complete experimental validation of the simulator. The generated flow, pressure, and volume profiles should be compared with reference measurements over the entire intended operating range. Repeated trials should be performed for each operating mode to quantify accuracy, repeatability, settling time, waveform-tracking error, and cycle-to-cycle variability.

The calibration of the custom pneumotachograph should also be extended to a wider range of positive and negative flows. Its measurements should be systematically compared with those of the commercial Sensirion flowmeter, and the resulting calibration relationship should be stored in non-volatile memory and automatically applied by the firmware.

Further work is required to optimize the PID controller. The controller parameters should be tuned and validated for different respiratory amplitudes, frequencies, mechanical loads, and directions of motion. Particular attention should be given to the aspiration phase, in which friction, pressure losses, and mechanical resistance produced greater actuation difficulties. The predictive contribution included in the control strategy should also be quantitatively evaluated to determine its effective improvement in waveform tracking.

From a mechanical perspective, temporary solutions such as tape, manually drilled holes, and adjusted printed components should be replaced with redesigned parts manufactured using appropriate tolerances and more resistant materials. The syringe and encoder supports could be further reinforced, while the pneumatic path should be redesigned to reduce leakage, dead volume, unwanted resistance, and flow disturbances. Long-duration endurance tests would also be useful to evaluate wear of the transmission system, couplings, syringe, and printed components.

The second PCB revision should be fabricated and experimentally validated. Particular attention should be given to power integrity, signal integrity, I²C communication, encoder level translation, motor-driver interfacing, electromagnetic interference, and thermal behaviour. The revised board should also be tested under motor start-up, direction reversal, emergency-stop, and fault conditions.

Additional safety improvements could include redundant position monitoring, a fully validated and mandatory start-up homing and self-test procedure, improved management of hardware and software limits, pressure and flow thresholds, communication timeouts, and automatic motor shutdown under abnormal operating conditions.

Finally, the external software could be expanded with improved real-time visualization, automatic test execution, calibration tools, data logging, and quantitative comparison between desired and measured waveforms. A larger library of physiological and pathological respiratory

patterns could also be introduced, allowing the simulator to reproduce more realistic in vivo signals and increasing its usefulness for the development and evaluation of respiratory monitoring devices.